

Prueba Sumativa N° 2

Programación – 26 de junio de 2024

Nombre		RUT	
Paralelo	C. Galleguillos C1 () C. Rabi C2 () A. Bordones C3 () F. Tapia C4 () F. Guerra C5 () F. Quiroz C6 () A. Paolini C7 ()		

Antecedentes generales:

Puntaje total de la prueba	1 punto	Puntos para nota aprobatoria	0.6 puntos	Duración de la prueba	3 horas
Resultados de Aprendizaje a evaluar	RA2. Construir programas que satisfagan las especificaciones, verificando el comportamiento esperado de las soluciones implementadas. RA4. Identificar los objetivos y requerimientos de las soluciones TIC. RA5. Desarrollar la solución tecnológica más adecuada en base a las características del problema y los recursos disponibles.				
Fecha de entrega de resultados	3 de julio				

Instrucciones:

- Esta evaluación tiene 7 páginas y 2 problemas. Compruebe que dispone de todas las páginas.
- Durante la prueba no se puede utilizar: teléfonos móviles, apuntes y/o computador (exceptuando el uso de Spyder). Está prohibido intentar conectarse a internet de cualquier manera. Si es sorprendido obtendrá la calificación mínima. Tampoco puede utilizar dispositivos de almacenamiento externos o cualquier otro dispositivo como relojes inteligentes, audífonos, etc.
- Lea la prueba completamente DOS veces antes de hacer cualquier pregunta.**
- Una prueba respondida correctamente en un 60%, de acuerdo a las ponderaciones asignadas, corresponde a una nota 4,0.
- La prueba es individual, cualquier sospecha de copia será calificada con la nota mínima y el caso será remitido al comité de ética. Si leyó hasta este punto, felicidades, para saber que lo hizo dibuje una galleta al final de esta página.
- Recuerde colocar su nombre en una de las primeras 5 línea de código de cada archivo, de lo contrario tendrá un descuento del 25% en cada problema que no lo tenga.
- En su espacio personal no debe haber nada más que hojas de papel en blanco, lápiz, goma y/o calculadora.
- El resto de sus implementos debe guardarlos dentro de su mochila/bolso y ésta debe posicionarse al frente debajo de la pizarra.

Acepto las condiciones firmando: _____

Un clicker (también conocido como incremental game o idle game) es un tipo de videojuego en el que el jugador realiza acciones repetitivas, como hacer clic, para acumular una moneda o recurso virtual, y luego usa esos recursos para automatizar la producción y progresar más rápido.

En esta oportunidad haremos un clicker de galletas en Python. Solo que los clics los reemplazamos por pulsar la tecla “Enter”, cada vez que pulsamos la tecla se generará una cantidad determinada de galletas. Inicialmente, por cada Enter pulsado se genera 1 galleta, que es el equivalente a trabajar sin ningún tipo de artefacto extra.

El usuario tiene la opción de vender sus galletas cuando desee para obtener monedas a cambio, estas son necesarias para comprar artefactos que permiten generar más galletas por pulsación. Para hacer funcionar estos artefactos se requieren monedas, cada uno tiene un coste de uso específico por pulsación. Por ejemplo, supongamos que tenemos el artefacto “Horno”, por cada clic produce 5 galletas pero cuesta 1 moneda para hacerlo funcionar. Ahora bien, el jugador tiene permitido tener varios artefactos a la vez, supongamos que esta vez compró 2 artefactos:

- Horno, genera 5 galletas por pulsación, cuesta 1 moneda hacerlo funcionar
- Horno industrial, genera 10 galletas por pulsación, cuesta 6 monedas hacerlo funcionar

Entonces por cada pulsación se generan: 1 + 5 + 10 galletas (el 1 es la generación base), pero además se gastan 1 + 6 monedas.

Configuración

La configuración del juego viene en un archivo llamado “config.txt”. Acá está la cantidad de artefactos máxima que puede tener el jugador, el valor en monedas de cada galleta y todos los artefactos disponibles en la tienda. Tiene el siguiente formato:

Cantidad Max Artefactos
Precio de una galleta
Nombre Artefacto 1,Precio,Cantidad de Galletas que genera por Pulsación,Coste por clic
Nombre Artefacto 2,Precio,Cantidad de Galletas que genera por Pulsación,Coste por clic
...

Donde:

- Cantidad máxima de Artefactos:** Son los slots disponibles que tiene en su fábrica
- Precio de una galleta:** Es el precio unitario de venta de una galleta
- Nombre Artefacto:** Es el nombre del dispositivo
- Precio:** Valor en monedas que se paga para obtenerlo
- Cantidad de Galletas que genera por pulsación:** Número de galletas que genera por pulsación
- Coste por clic:** Monedas que se gastan por cada pulsación

Ejecución

Al ejecutar el programa, el juego comienza inmediatamente hasta que el usuario ingrese “FIN”. Finalmente se muestran unas estadísticas de la partida y se acaba la ejecución. Durante el juego siempre se muestra al usuario la cantidad de galletas y monedas que posee, además de un menú, el cual permite:

Vender Galletas:

Pregunta cuántas galletas quiere vender, luego se muestra la cantidad de monedas obtenidas

```
Galletas: 60 | Monedas: 0 | Galletas por pulsación: 1 | Coste total: 0
1. Vender Galletas
2. Ver Artefactos
3. Comprar Artefactos
4. Vender Artefactos
ENTER para producir | 'FIN' para salir
```

```
>>> 1
¿Cuántas galletas quieres vender? 60
Vendiste 60 galletas por 300 monedas.
```

Ver Artefactos

Muestra todos los artefactos, su nombre, precio, galletas por pulsación, coste operacional en monedas, coste por galleta, el cual se calcula así:

Coste en monedas por la pulsación / Cantidad de Galletas producidas por pulsación

```
Galletas: 0 | Monedas: 160 | Galletas por pulsación: 151 | Coste total: 4
1. Vender Galletas
2. Ver Artefactos
3. Comprar Artefactos
4. Vender Artefactos
ENTER para producir | 'FIN' para salir
>>> 2

--- Tus Artefactos ---
Horno eléctrico | Precio: 40 | Galletas/Pulsación: 50 | Coste: 1 | Coste/Galleta: 0.02
Horno a gas | Precio: 100 | Galletas/Pulsación: 100 | Coste: 3 | Coste/Galleta: 0.03
```

Comprar Artefactos

Se muestra la tienda de Artefactos junto con el saldo disponible en monedas y la cantidad de slots disponibles, se le pregunta al jugador cuál Artefacto quiere comprar

```
=== Comprar Artefacto ===
1. Horno eléctrico | Precio: 40 | Galletas/Pulsación: 50 | Coste: 1
2. Horno a gas | Precio: 100 | Galletas/Pulsación: 100 | Coste: 3
3. Horno industrial | Precio: 250 | Galletas/Pulsación: 150 | Coste: 10
4. Gato amasador | Precio: 500 | Galletas/Pulsación: 200 | Coste: 5
5. Fábrica pequeña | Precio: 10000 | Galletas/Pulsación: 1000 | Coste: 150
6. Fábrica grande | Precio: 100000 | Galletas/Pulsación: 5000 | Coste: 400
7. Horno del Tío Cochino | Precio: 100000000000 | Galletas/Pulsación: 1000000 | Coste: 100

Monedas disponibles: 4169
Espacio disponible: 2 artefactos
¿Cuál deseas comprar? (número) 1
Compraste Horno eléctrico.
```

Vender Artefactos

Se muestran todos los artefactos y se pregunta al usuario cuál desea vender. El artefacto se vende al 90% del precio de compra.

```
Galletas: 0 | Monedas: 160 | Galletas por pulsación: 151 | Coste total: 4
1. Vender Galletas
2. Ver Artefactos
3. Comprar Artefactos
4. Vender Artefactos
ENTER para producir | 'FIN' para salir
>>> 4

=== Vender Artefacto ===
--- Tus Artefactos ---
1. Horno eléctrico | Precio Compra: 40
2. Horno a gas | Precio Compra: 100
¿Cuál deseas vender? (número) 1

Vendiste Horno eléctrico por 36 monedas.
```

Si el usuario no desea hacer nada del menú, simplemente pulsa Enter y se genera la cantidad de galletas que corresponda. Se debe mostrar siempre la cantidad de galletas que posee, las monedas, cuántas galletas se generan por pulsación y el coste en monedas de cada momento. El menú siempre debe estar disponible.

Por cada pulsación se genera la siguiente cantidad de galletas:

$$\text{Cantidad de Galletas por Pulsación} = 1 + \sum \text{Cant generada por el artefacto}_i$$

En otras palabras, se genera la galleta base (1) más las que generan todos los artefactos (la sumatoria). Por ejemplo, si tengo 2 artefactos que producen 10 y 20 galletas por pulsación respectivamente, al hacer una pulsación, se generan: 1 + 10 + 20 galletas.

Reporte final

1. La cantidad máxima de galletas producida en una pulsación
2. Cantidad de galletas generada durante toda la partida
3. Cantidad de monedas gastadas en producir galletas durante toda la partida (no en compra de artefactos)
4. Mostrar los artefactos de la Fábrica al momento de finalizar de menor a mayor según el costo por galleta (monedas/galleta)

```
>>> FIN

=== REPORTE FINAL ===
Máx. galletas por pulsación: 351
Galletas totales generadas: 969
Monedas totales gastadas: 680
Artefactos ordenados por coste por galleta:
Horno eléctrico | Coste por galleta: 0.02
Gato amasador | Coste por galleta: 0.03
Horno a gas | Coste por galleta: 0.03
```

Consideraciones

- No se puede vender más galletas de las que se tienen
- El jugador nunca debe quedarse con una cantidad negativa de monedas, por lo tanto, si en la pulsación se gastarían más monedas de las que se tienen, se debe avisar y no fabricar galletas, ni siquiera la base (1). (Ver “salida restricciones.txt”)
- Si se compra un artefacto de la tienda, este sigue estando disponible en ella. Los artefactos tienen “stock infinito”
- El uso de listas es obligatorio, si no utiliza listas no se revisará el ejercicio
- Cada artefacto nuevo se agrega al final de la secuencia de artefactos. Al vender uno, no se dejan espacios entre ellos.
- Si en los menús se ingresa un valor inválido, simplemente se vuelve al menú principal, no se vuelve a pedir el valor
- El archivo de ejemplo junto con la salida está en Campus Virtual. Está resumido, debido a que la salida es extensa por las pulsaciones. Hay un segundo archivo de salida que incluye casos especiales
- No es necesario que pulse el botón Enter repetidas veces, basta con que lo mantenga presionado

Problema 2. Batalla Naval: Simulación de Ataques con Misiles (50%)

La simulación de batalla naval consiste en un enfrentamiento entre dos jugadores que disponen cada uno de un tablero donde están ubicadas sus "naves" o posiciones protegidas. El objetivo es destruir todas las naves del adversario mediante una serie de ataques con distintos tipos de misiles que tienen diferentes patrones de impacto.

El programa debe leer desde un archivo batallas.txt la información de varias batallas, que incluyen la configuración inicial de los tableros y la secuencia de movimientos de ataque realizados por ambos jugadores. La simulación debe evaluar cada movimiento, marcar los impactos, contabilizar daños y validar reglas específicas para el uso de misiles especiales.

Formato del archivo batallas.txt

La primera línea indica la cantidad de batallas a simular, un número entero positivo.

Para cada batalla se incluyen los siguientes datos:

- 1. Línea con el nombre del jugador 1, nombre del jugador 2, y la cantidad total de movimientos (separados por comas).
- 2. Línea con las dimensiones del tablero: número de filas y columnas (separados por coma).
- 3. Las siguientes N líneas: tablero del jugador 1, con valores separados por comas:
 - 1 → posición con nave.
 - 0 → posición vacía (agua).
- 4. Luego, otras N líneas para el tablero del jugador 2 (mismo formato).
- 5. Bloque de líneas con los movimientos, uno por línea, con los datos:
 - Nombre del jugador que realiza el movimiento.
 - Tipo de misil usado (normal, lineal, cruz, x).
 - Largo o alcance del misil (entero).
 - Coordenada X (fila) del objetivo (entero, base 0).
 - Coordenada Y (columna) del objetivo (entero, base 0).
 - Los valores van separados por comas.

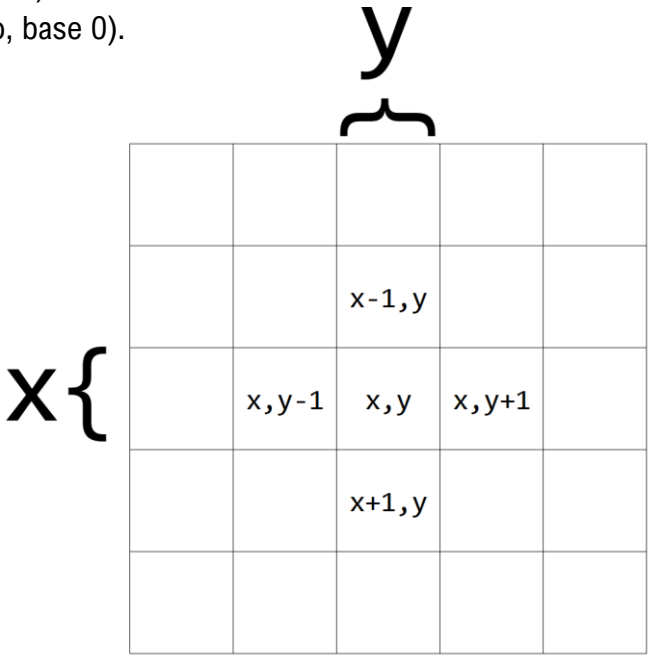
Ejemplo de archivo

1
Ana, Leo, 3
2, 3
1, 0, 1
0, 0, 1
1, 1, 1
0, 1, 0
Ana, normal, 1, 0, 0
Leo, lineal, 1, 1, 1
Ana, cruz, 1, 1, 2

Representación del tablero

Cada posición del tablero puede contener:

- 1 → nave
- 0 → agua
- X → impacto exitoso (antes era 1 y como sufre un impacto pasa a ser -1)
- 0 → impacto fallido (antes era 0 y como sufre un impacto pasa a ser -2)



Tipos de misiles y su patrón de impacto

- **Normal:** Afecta solo la posición objetivo (x, y) .
- **Lineal:** Afecta la posición objetivo y las posiciones horizontales a la izquierda y derecha hasta el alcance (largo). Ejemplo con $\text{largo}=2$ afecta: $(x, y-2)$, $(x, y-1)$, (x, y) , $(x, y+1)$, $(x, y+2)$.
- **Cruz:** Afecta la posición objetivo y las posiciones en cruz vertical y horizontal hasta el alcance largo. Ejemplo con $\text{largo}=1$ afecta: (x, y) , $(x-1, y)$, $(x+1, y)$, $(x, y-1)$, $(x, y+1)$.
- **X:** Afecta la posición objetivo y las posiciones diagonales en forma de "X" hasta el alcance largo. Ejemplo con $\text{largo}=1$ afecta: (x, y) , $(x-1, y-1)$, $(x-1, y+1)$, $(x+1, y-1)$, $(x+1, y+1)$.

Restricciones y reglas del juego

- Cada jugador puede usar un solo misil especial (lineal, cruz, o x) por batalla. Si intenta usar más de uno, se considera trampa y el movimiento se ignora.
 - a) Intentos de trampa: Se deben contar y reportar los intentos de usar misil especial repetido.
- Los impactos:
 - a) Si se acierta una nave (1), la casilla pasa a -1.
 - b) Si se falla (era 0), la casilla pasa a -2.
- Las coordenadas fuera del tablero deben ser ignoradas sin afectar el juego.
- La batalla termina:
 - a) Si un jugador pierde todas sus naves.
 - b) si se ejecutan todos los movimientos permitidos.

Condiciones de victoria

- Victoria por destrucción total: Un jugador gana si destruye todas las naves del adversario antes de que se acaben los movimientos.
- Victoria por mayor impacto: Si se terminan los movimientos y ambos jugadores tienen naves, gana quien tenga mayor cantidad de impactos exitosos.
- Empate: Si ambos jugadores tienen la misma cantidad de impactos exitosos.

Salida esperada

Al ejecutar el programa, para cada batalla debe mostrar:

- El tablero inicial de ambos jugadores.
- El resultado de cada movimiento con impactos y actualización del tablero afectado.
 - Para mostrar el tablero puede usar la siguiente función:

```
def imprimir_matriz(m):  
    for fila in m:  
        print(' '.join(['1' if val == 1 else '-' if val == 0  
                        else 'X' if val == -1 else '0' for val in fila]))  
        print()
```

- Mensajes de intentos de trampa o ataques fuera de rango.
- Mensaje indicando si la batalla terminó porque un jugador pierde todas sus naves.
- Resultado final indicando:
 - Ganador o empate.
 - Impactos totales por jugador.
 - Intentos de trampa por jugador.
 - Estado final de los tableros.

```
=== Batalla 1 ===
Tablero inicial de Ana:
1 - 1
- - 1

Tablero inicial de Leo:
1 1 1
- 1 -

Movimiento 1: Jugador Ana usó 'normal' en (0,0), impactos: 1
Tablero objetivo tras movimiento:
X 1 1
- 1 -

Movimiento 2: Jugador Leo usó 'lineal' en (1,1), impactos: 1
Tablero objetivo tras movimiento:
1 - 1
0 0 X

Movimiento 3: Jugador Ana usó 'cruz' en (1,2), impactos: 2
Tablero objetivo tras movimiento:
X 1 X
- X 0

=== Resultado de la batalla ===
Ana: impactos = 3, trampas = 0
Leo: impactos = 1, trampas = 0
Ganador por daño: Ana
```

Ejemplo de ejecución con el archivo txt entregado para la prueba:

Encontrarás la salida completa entre los archivos del campus virtual.